

Using Kernel Density Function and Student's t-distribution to model Value at Risk and Expected Shortfall using Monte Carlo method

Gianni Mazaud
gianni.mazaud@icloud.com

Abstract—Basel III regulation imposes a clear market risk frameworks for financial institutions. The principal shift rests in the core of the attention now turned toward a 97.5% Expected Shortfall, rather than VaR at 99% via FRTB.

In this paper we use Monte Carlo simulations through Student's t-distribution and non-parametric Kernel Density Estimator (KDE) to calculate VaR and CVaR at the 99% and 97.5% threshold.

Empirically, our simulations demonstrate that the Geometric Brownian Motion framework penalizes risk architecture by collapsing the threshold difference between Value at Risk and Expected Shortfall. In contrast, both the parametric Student's t-distribution and the non-parametric Kernel Density Estimator successfully capture extreme market shocks with fatter tail distributions. In each scenario, we observe that $CVaR(97.5\%)$ is greater than $VaR(99\%)$, validating the Basel III Fundamental Review of the Trading Book (FRTB) guidelines.

We demonstrate empirically that KDE captures the asymmetric tail structure of real returns that Student's t cannot, with implications for Basel III capital adequacy.

Index Terms—Value at Risk, Expected Shortfall, Monte Carlo modeling, Kernel Density Estimator, Student's t-distribution, Asymmetric tail risk

I. INTRODUCTION

Value at Risk is defined as: "a measure of the worst expected loss on a portfolio of instruments resulting from market movements over a given time horizon and a pre-defined confidence level" [1] [2]. Formally, it can be written as follow:

$$VaR_\alpha = \inf \{l \in \mathbb{R} : \mathbb{P}(L > l) \leq 1 - \alpha\}$$

Expected Shortfall or Conditional Value at Risk is defined as: "a measure of the average of all potential losses exceeding the VaR at a given confidence level" [1]. Mathematically, we can write it as:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha]$$

Under the Basel III regulation, the confidence level for VaR is $\alpha = 99\%$, even though it focuses primarily on Expected Shortfall. Developed by the Basel Committee on Banking Supervisions (BCBS), this regulation was first implemented in July 2013 and finally reformed in December 2017. Its first aim was to respond to the 2007-2008 global financial crisis. To counter these risks it mainly focuses on enhanced capital requirements, non-risk-based leverage ratio, and counterparty credit risk.

In terms of market risk management, Basel III —specifically through the Fundamental Review of the Trading Book (FRTB)— shifts the core metric from a 99% Value at Risk (VaR) to a 97.5% Expected Shortfall (ES). This shift implies that regulators no longer satisfied with knowing just the threshold of a bank's potential trading losses, but rather the actual severity of losses when a tail-risk event occurs. Additionally, while the previous VaR framework assumed a static, short-term liquidity horizon across all asset classes, the new CVaR-based framework forces banks to map different liquidity horizons to different risk factors.

Banks aren't talkative on the subject and it's hard to really understand, when they are using the Monte Carlo method, which diffusion model they use [3]. Our goal here will be to compare three different approaches. First, we will use a Geometric Brownian motion, that assumes the logarithm of asset prices follows a Brownian motion with a drift. It implies that the probability of extreme events is highly underestimated. In order to offset that, a Student's t-distribution will be implemented to take into account fat-tail risks. This model is usually a better fit for financial returns and it captures extreme events more accurately. Finally, we are going to use a Kernel Density Estimator which is a non-parametric way to estimate the true distribution of financial returns directly from data.

II. USUAL METHODS

Parametric method

Generally, Basel III allows for three methods: parametric, historic, and Monte Carlo.

Parametric method simply assumes loss follows a Gaussian (normal) distribution, then we have: $L \sim N(\mu_L, \sigma_L^2) \Rightarrow L = \mu_L + \sigma_L U$, with $U \sim N(0, 1)$.

Under this method, the formulas are:

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$\begin{aligned} CVaR_\alpha &= \mathbb{E}[L|L \geq VaR_\alpha] \\ &= \frac{1}{1 - \alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1 - \alpha} \end{aligned}$$

(See Appendix 1. for demonstrations.)

Historical method

This method rests on the assumption that the worst case scenarios of the period taken into account will remain the same. Nevertheless, reading [3], we can see that the distribution can still be adapted following some scenarios. Furthermore, it assumes portfolio remains the same, the weight of each asset being unchanged.

Let $r_{i,t}$ be the daily returns for each asset i at date $t \in [i, T]$ our historical interval. We apply today's weight w to each historical return scenario:

$L_t = -\sum_{i=1}^T \omega_i r_{i,t} = -w^T r_t$ (it takes the opposite value since loss is considered the opposite of the asset return). It gives us T realized loss scenarios, using the current portfolio composition replayed through history. We order the T losses in ascending order: $L_{(1)} < L_{(2)} < \dots < L_{(T)}$.

Finally, we have:

$$VaR_\alpha = L_{[\alpha, T]}$$

$$CVaR_\alpha = \frac{1}{T - \alpha T} \sum_{t=\alpha T+1}^T L_{(t)}$$

III. MONTE CARLO METHOD

Method framework

The Monte Carlo method is a way to find answers to hard financial problems by using random numbers and simulating thousands of independent paths following the same distribution. First, we choose a statistical rule for how our assets behave (for instance a log-normal distribution or a Student's t-distribution). It allows us to see where the stock prices can potential go. Finally, we put all these simulated results together from the worst loss to the biggest profit. This gives us a huge list of possibilities, and we just look at the bottom 1% of the worst results to easily find the Value at Risk (VaR) or take the average of those bad results to get the Conditional VaR (CVaR).

Statistical distributions

Our goal now is to implement and compare three different statistical distributions: log-normal, Student's t, and Kernel Density Estimator.

Log-normal distribution

Geometric Brownian motion makes sure prices are strictly positive by using an exponential. Also, it scales volatility and drift based on the current price. Finally, this model assumes that continuously compounded returns are normally distributed. If we assume that an asset S follows a log-normal distribution, we have:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where S_t is the asset price at time t , μ the drift (expected return), σ the volatility coefficient, dt an infinitesimal time step,

dW_t a Wiener process (or Brownian motion) increment, which captures the random market shocks where $dW_T \sim N(0, dt)$ [4]. It has the following properties:

- 1) $W_0 = 0$.
- 2) It has independent increments.
- 3) For any two time steps s and t (where $t > s$), the change in value is normally distributed: $W_t - W_s \sim N(0, t - s)$.

The analytical solution, by applying Itô's Lemma [5] to solve the SDE, gives:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \right)$$

(Demonstrations are shown in Appendix 2.)

If we set $s = 0$ and look strictly at a single future maturity date $t = T$, the total increment is:

$$W_T - W_0 = W_T \sim N(0, T)$$

If we scale a standard normal random variable $Z \sim N(0, 1)$ by multiplying it by $\sqrt{T}$, its mean remains 0 and its variance becomes $(\sqrt{T})^2 = T$. Therefore:

$$W_T \stackrel{d}{=} Z\sqrt{T}$$

Since W_T can be written as $Z\sqrt{T}$, where $Z \sim N(0, 1)$, the formula used is:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma Z\sqrt{T} \right)$$

Student's T-distribution [6]

The Student's t-distribution does not only rely on the mean and variance, but introduces a third parameter: degrees of freedom (ν). Further in this paper, we use $\nu = 4$ as a conservative choice to ensure fat-tailed behavior while maintaining finite variance. The mathematical formula for the Probability Density Function (PDF) of such a distribution is:

$$f(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu} \right)^{-\frac{\nu+1}{2}}$$

Where the Gamma function is defined as:

$$\Gamma(z) = \int_0^{+\infty} t^{z-1} e^{-t} dt$$

If the degrees of freedom is low ν the distribution has very fat tails and a higher peak (leptokurtic). However when $\nu \rightarrow \infty$ the distribution mathematically converges into the standard normal distribution.

Modeling an asset using this distribution, we get:

$$S_T = S_0 \exp \left(\mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T} \right)$$

Where T_ν is a random variable drawn from a standard Student's t-distribution with ν degrees of freedom and $\sqrt{\frac{\nu-2}{\nu}}$ is the scaling factor ensuring that the variance of the return over

time matches our target volatility. (Proof is given in Appendix 3.)

Kernel Density Estimator

The formal mathematical formula for a Kernel Density Estimator is:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

Where h is the bandwidth, and $K(\cdot)$ the Kernel function. If we use the Gaussian (Normal) Kernel: $K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2}$, we finally get:

$$\hat{f}_h(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n e^{-\frac{1}{2}\left(\frac{x-x_i}{h}\right)^2}$$

the summation gives us exactly the probability density function (PDF) of a normal distribution $N(x_i, h^2)$. Therefore, the KDE can be written as:

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n f_N(x|\mu = x_i, \sigma^2 = h^2)$$

This tells us that the KDE treats the asset return distribution as a uniform mixture of n distinct normal distributions. Assuming that $y_j \sim N(x_i, h^2)$ we have that:

$$y_j = x_i + hZ_j$$

To decide which value to choose for h we will use the Silverman's Rule of Thumb [7], that states:

$$h = \left(\frac{4\hat{\sigma}^5}{3n}\right)^{\frac{1}{5}} \approx 1.05\hat{\sigma}n^{-\frac{1}{5}}$$

The core difference of this approach compared to a Brownian motion is that it assume no prior functional form for the distribution. Instead of forcing the data into a bell curve, the data defines its own shape.

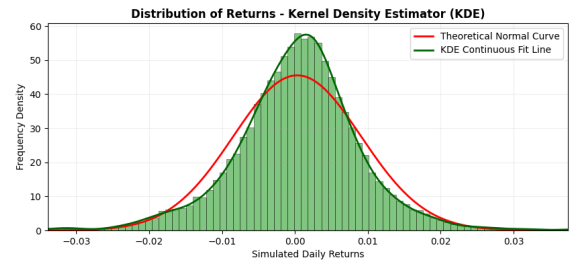
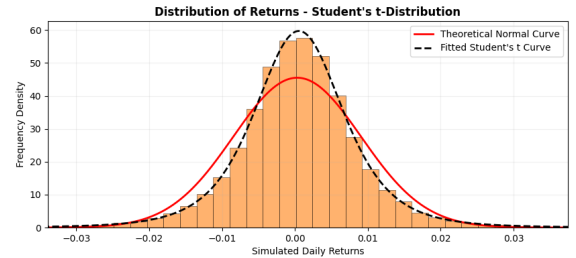
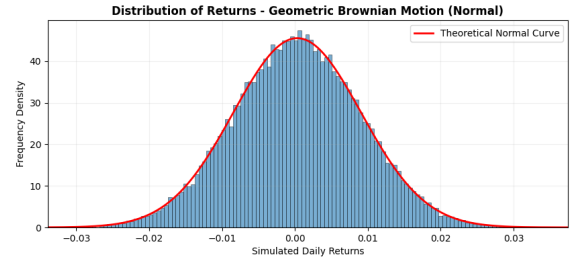
How to decide which distribution model to use?

KDE (Non-Parametric) does not assume any underlying shape for our data. It builds a distribution directly from historical data points by placing a "kernel" (smooth curve) over each observation. It is purely empirical. Student's t (Parametric) assumes a fixed mathematical shape defined by specific parameters (mean, scale, and degrees of freedom). It is symmetrical but features heavy tails, making it excellent for capturing extreme financial shocks. GBM (Stochastic Process) unlike the other two, GBM is not a static probability distribution—it is a time-dependent path generator. It models how an asset price moves over time based on a constant drift (trend) and volatility (random walk).

IV. IMPLEMENTATIONS & RESULTS

Using Python, we can simulate an asset N times and adjust the characterizing parameters as required for our study. The python code can be found in Appendix 4. For our study, we construct an equally-weighted portfolio made of: S&P

500, NASDAQ, CAC40, FTSE 100, DAX, and Hang Seng. This portfolio is of course a simplified version of what could be seen in practice but we assume for a CVaR and VaR simulation it can fit. We extract the data for a five years period, from 06/01/2021 to 06/01/2026, with yfinance module in Python. The prices used are daily close prices and we consider log-returns since it assumes symmetry around zero which is useful for modeling. Mean and standard deviation are computed via a Maximum Likelihood estimation which provides a uniform, consistent framework to compare the normal distribution against non-Gaussian models (like the Student's t -distribution) that require statistical fitting rather than simple averages. After gaining these data, we model 50,000 paths for each distribution. This presents some assumptions and simplifications. In fact, real trading books re-balance often which is not the case here in our model. To get a better picture of how returns are distributed, here are the histograms representing the distribution of our generated-data.



The Geometric Brownian Motion (GBM) model assumes that asset returns follow a perfect normal distribution. In our simulation, the GBM skewness yields a z-score of -0.8703 ($p = 0.384$), and the excess kurtosis yields a z-score of

TABLE I: Statistical Significance Tests for Skewness and Kurtosis

Model	Test / Metric	Value / Stat	z-score	p-value
GBM	Skewness	-0.0095	-0.8703	0.384119
	Ex. Kurtosis	+0.0010	+0.0465	0.962946
Student's t	Skewness	+0.0004	+0.0330	0.973636
	Ex. Kurtosis	+14.7932	+675.2208	0.000000***
KDE	Skewness	-1.0377	-94.7309	0.000000***
	Ex. Kurtosis	+11.2080	+511.5783	0.000000***

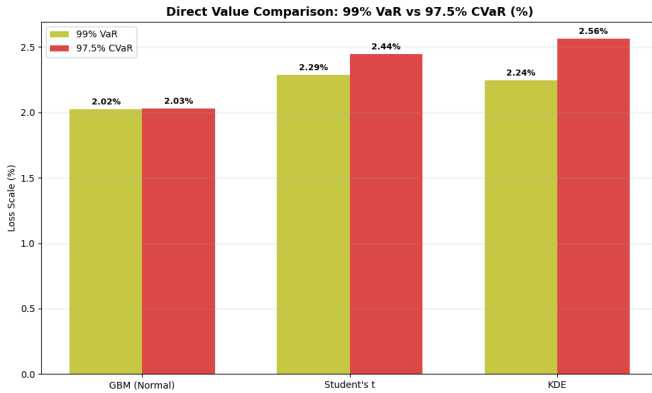
Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

+0.0465 ($p = 0.963$). Because both p-values are much larger than 0.05, these results are not statistically significant, meaning the distribution is perfectly Gaussian. Consequently, the GBM model is completely blind to unexpected market shocks or severe tail risks, making it an unrealistic tool for downside risk management.

To fix the limitations of the normal model, a Student's t-distribution was fitted to the data. While the skewness remains statistically insignificant ($z = +0.0330$, $p = 0.974$), the excess kurtosis shifts drastically, showing an extreme z-score of +675.2208 ($p < 0.001$). This highly significant p-value proves that the asset's returns have much fatter tails than a normal distribution. On the chart, the Student's t curve successfully captures the heavy risk in the tails, making it a much more reliable model for calculating portfolio Value-at-Risk (VaR).

Although the Student's t-distribution models fat tails well, its mathematical structure forces the data to be perfectly symmetric. The non-parametric Kernel Density Estimator (KDE) removes this restriction and reveals a highly significant negative skewness with a z-score of -94.7309 ($p < 0.001$), alongside a significant excess kurtosis z-score of +511.5783 ($p < 0.001$). These significant p-values confirm that the distribution is heavily distorted to the left. This proves the true asymmetry of financial risk: asset returns typically face sudden, severe negative crashes rather than positive jumps.

Finally, putting altogether our estimated risk measures:

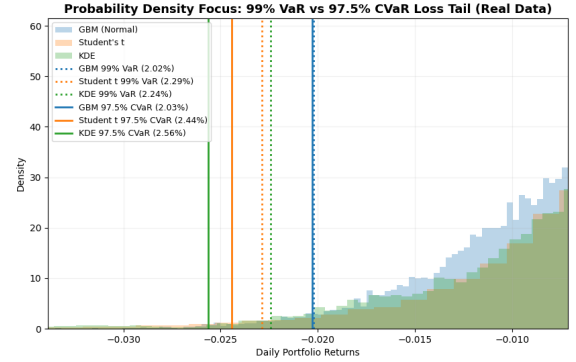


This histogram shows a big difference between the models.

Under the normal GBM model, the $VaR(99\%) = 2.02\%$ and the $CVaR(97.5\%) = 2.03\%$ are almost exactly the same, with a gap of only 0.01%. This happens because the normal model assumes that extreme losses are nearly impossible. In the real world, using this normal model is dangerous because it leaves a portfolio unprotected against big market drops.

When we look at the other models, the risk numbers increase to show the true dangers of the market. The Student's t-distribution gives a $VaR(99\%)$ of 2.29% and a $CVaR(97.5\%)$ of 2.44%, showing a larger gap of 0.15% because it captures fat tails. However, the KDE model gives the most realistic view of risk. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ jumps to the highest level at 2.56%. This 0.32% gap is caused by the negative skewness of the data, proving that real-world market crashes are much worse than simple models predict.

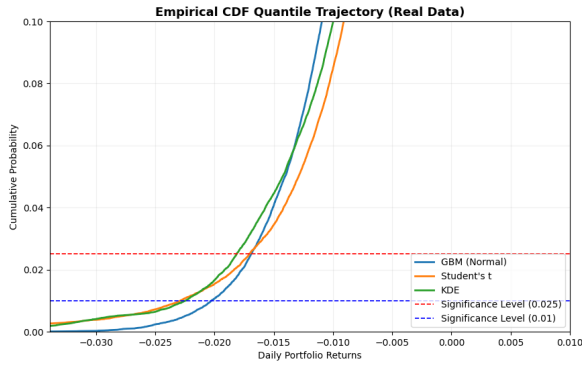
Also, we notice that in every case $CVaR(97.5\%)$ is above $VaR(99\%)$ reflecting the reinforcement in capital exposition regulation imposed by Basel III.



The graph of Probability Density shows the worst-case losses for the three different models. The normal model (GBM) is represented by the blue lines. Here, the $VaR(99\%)$ is 2.02% and the $CVaR(97.5\%)$ is 2.03%. Because these two lines are almost on top of each other, the model assumes that there are no risk of extreme scenarios.

The other two models look at the actual danger in the market much better. As seen in the graph, the Student's t distribution (orange lines) separates the VaR (2.29%) and CVaR (2.44%) because it takes into consideration the market has fat tails. However, the KDE model (green lines) shows the most realistic danger. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ moves way to the left at 2.56%. This large gap happens because real financial data has a negative skew, meaning that when markets crash, they fall fast and deep.

$CVaR(97.5\%)$ is always represented at the left (deeper into high-loss territory). Unlike the normal distribution, KDE and Student's t-histogram bars continue to exist on the left part of the graph. This illustrates again the reason why Basel III guidelines transitioned regulatory capital calculations to a $CVaR(97.5\%)$ framework.



The chart empirical CDF shows the cumulative probability of negative return. The two horizontal dashed lines represent the risk thresholds: the red line is the 2.5% risk level (used for 97.5% CVaR), and the blue line is the 1% risk level (used for 99% VaR). This chart shows exactly where each model intersects these risk lines.

As we look more closely, we can see how the lines cross the thresholds differently.

- The blue GBM (blue) line stays to the right of the others.
- However, both the Student's t (orange) and KDE (green) lines are much further to the left.

The fact that the orange and green curves sit higher up on the left side proves that large losses happen much more often in real life than the normal GBM model expects.

V. LIMITATIONS

Our entire simulation operates on a one-day return horizon. FRTB explicitly requires liquidity-adjusted horizons ranging from 10 days to 120 days. Our results therefore represent a best-case, most liquid scenario and would need to be scaled before being operationally used in a real regulatory context.

Our Monte Carlo draws each scenario as if the six indices move independently. In reality, tail dependence between equity markets is well documented: during the 2020 Covid crash or 2008 crisis, global indices fell simultaneously and correlations spiked toward 1. A copula-based approach would capture this.

The equal-weight assumption is held fixed throughout the whole time-period where we collect data. In reality, portfolio allocation evolves continuously which changes the loss distribution.

Silverman's rule-of-thumb is considered optimal for estimating the bulk of a distribution, but also it is known to over-smooth the tails. A smaller bandwidth would produce heavier tails and at the same time higher risk measures estimates.

The Student's t-distribution is symmetric by construction, meaning it assigns equal probability to extreme gains and extreme losses. Since financial returns exhibit asymmetric crash risk (negative skewness), the Student's systematically underestimates left-tail severity relative to a model that allows for skewness.

VI. CONCLUSION

This paper introduces three distributions to evaluate Value at Risk and Expected Shortfall under the Basel III (FRTB) regulation. We modeled 50,000 different random paths for each distribution in order to evaluate these risk measures.

The first distribution used is a geometric Brownian motion (using Gaussian distribution assumptions), and our findings show that it fails to capture fat-tail risk. Indeed, as calculated, the excess kurtosis is near zero showing the lack of consideration for extreme scenarios.

The second distribution implemented is the Student's t-distribution in order to overcome that issue of not considering fat-tail risk. This distribution exhibits a clearly positive excess kurtosis which implies a leptokurtic distribution as we can observe with the corresponding histogram. Finally, the Kernel Density Estimator also exhibits a leptokurtic distribution and a clearly lower skewness than the two others. As the data tells, this distribution gives the highest Expected Shortfall but still has a lower Value at Risk, relative to the thresholds imposed by the regulation. This model is different in the fact that it is non-parametric, it let the data defines its own form.

Our findings demonstrate the danger of normality assumption. As we showed, the Gaussian distribution does not take consideration of fat-tail risks that are essential in risk measurement. On the one side, data clearly shows the geometric Brownian motion exhibits lower values for both measures (Expected Shortfall and Value at Risk) and gives almost the value for both of them showing again the ignorance of extreme scenarios. On the other side, KDE as well as Student's t distributions have a much higher CVaR(97.5%) than VaR(99%) proving the utility of the Basel III regulation that shifts toward the first one in order to reinforce capital requirement.

A parametric approach assumes that the distribution of a portfolio returns follows a known statistical distribution equation. To capture fat tails with a parametric model, Gaussian distribution must be forgotten. In a non-parametric framework like the Kernel Density Estimator (KDE), we get rid of equations to use data directly. Then, it captures leptokurtosis naturally. This gives more accurate measures, especially for hidden structural risks (such as the negative skewness caused by market sell-offs).

REFERENCES

- [1] Basel Committee on Banking Supervision, "Explanatory note on the minimum capital requirements for market risk," 2019. [Online]. Available: https://www.bis.org/bcbs/publ/d457_note.pdf
- [2] J.-P. Laurent, "Modèle structurel du risque de crédit," http://laurent.jeanpaul.free.fr/Enseignement/GRF_2023-24_modele_structurel_du_risque_de_credit.pdf, 2023, accessed: 2026-06-04.
- [3] Société Générale Group, "2025 universal registration document: 2024 annual financial report," Société Générale, Universal Registration Document Form 297, March 2025, filed with the Autorité des Marchés Financiers (AMF) on March 12, 2025. [Online]. Available: <https://usprogram.socgen.com/files/297.pdf>
- [4] M. Csörgő, "Brownian motion-wiener process," *Canadian Mathematical Bulletin*, vol. 22, no. 3, pp. 257–279, 1979. [Online]. Available: <https://www.cambridge.org/core/journals/canadian-mathematical-bulletin/article/brownian-motion-wiener-process/17DB3F7FE5C4F6216E6A214D166D44A6>

- [5] K. Itô, “On stochastic differential equations,” *Memoirs of the American Mathematical Society*, vol. 4, pp. 1–51, 1951, cited by: 3105. [Online]. Available: <https://doi.org/10.1090/memo/0004>
- [6] Student, “The probable error of a mean,” *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908.
- [7] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, ser. Monographs on Statistics and Applied Probability. London: Chapman and Hall, 1986, vol. 26.

VII. APPENDICES

Appendix 1. — demonstrations for parametric method

VaR is such that [2]:

$$\mathbb{P}(L < VaR_\alpha) = \alpha$$

Since we assumed $L \sim N(\mu_L, \sigma_L)$:

$$\Leftrightarrow \mathbb{P}(\mu_L + \sigma_L U < VaR_\alpha) = \alpha$$

With $U \sim N(0, 1)$, We can simplify as:

$$\begin{aligned} \Leftrightarrow \mathbb{P}\left(U < \frac{VaR_\alpha - \mu_L}{\sigma_L}\right) &= \alpha \\ \Leftrightarrow \Phi\left(\frac{VaR_\alpha - \mu_L}{\sigma_L}\right) &= \alpha \end{aligned}$$

$$\Leftrightarrow \frac{VaR_\alpha - \mu_L}{\sigma_L} = \Phi^{-1}(\alpha) \Leftrightarrow VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$CVaR_\alpha = \mathbb{E}[L | L \geq VaR_\alpha] = \frac{1}{1 - \alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl$$

with f_L the probability density function of $L \sim N(\mu_L, \sigma_L^2)$. Hence:

$$f_L(l) = \frac{1}{\sqrt{2\pi\sigma_L^2}} \exp\left(-\frac{(l - \mu_L)^2}{2\sigma_L^2}\right)$$

Let

$$\begin{aligned} \varphi : \frac{l - \mu_L}{\sigma_L} &\rightarrow \frac{1}{\sqrt{2\pi}} \exp\left(-\left(\frac{l - \mu_L}{\sigma_L}\right)^2 / 2\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(\frac{-(l - \mu_L)^2}{2\sigma_L^2}\right) \end{aligned}$$

This gives us:

$$f_L(l) = \frac{1}{\sigma_L} \varphi\left(\frac{l - \mu_L}{\sigma_L}\right)$$

Finally, we can compute the Expected Shortfall:

$$CVaR_\alpha(L) = \frac{1}{1 - \alpha} \int_{VaR_\alpha(L)}^{+\infty} \frac{l}{\sigma_L} \varphi\left(\frac{l - \mu_L}{\sigma_L}\right) dl$$

Let's define:

$$z = \frac{l - \mu_L}{\sigma_L} \Leftrightarrow l = \mu_L + \sigma_L z \Rightarrow dl = \sigma_L dz$$

This gives us:

$$\frac{1}{\sigma_L} \varphi\left(\frac{l - \mu_L}{\sigma_L}\right) \sigma_L dz = \varphi(z) dz$$

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

$$z = \frac{l - \mu_L}{\sigma_L} = \frac{(\mu_L + \sigma_L \Phi^{-1}(\alpha) - \mu_L)}{\sigma_L} = \Phi^{-1}(\alpha)$$

Finally:

$$CVaR_\alpha(L) = \frac{1}{1 - \alpha} \int_{\Phi^{-1}(\alpha)}^{+\infty} (\mu_L + \sigma_L z) \varphi(z) dz$$

$$CVaR_\alpha = \frac{1}{1 - \alpha} \left(\mu_L \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz + \sigma_L \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz \right)$$

Let's compute separately these two integrals:

$$I_1 = \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz = 1 - \Phi(\Phi^{-1}(\alpha)) = 1 - \alpha$$

$$\begin{aligned} I_2 &= \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz \\ &= - \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi'(z) dz = - \left[\varphi(z) \right]_{\Phi^{-1}(\alpha)}^{+\infty} = \varphi(\Phi^{-1}(\alpha)) \end{aligned}$$

We use here two arguments:

- $\varphi'(z) = -z\varphi(z)$
- $\varphi(z) \rightarrow 0$ when $z \rightarrow +\infty$

Hence:

$$\begin{aligned} CVaR_\alpha &= \frac{1}{1 - \alpha} [\mu_L(1 - \alpha) + \sigma_L \varphi(\Phi^{-1}(\alpha))] \\ \Leftrightarrow CVaR_\alpha &= \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1 - \alpha} \end{aligned}$$

Appendix 2. — Brownian motion

We let the following formula:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Let: $f(S_t) = \ln(S_t)$. For a continuous function $f(t, S_t)$ the second-order Taylor expansion using Itô's Lemma [5] gives us:

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} (dS_t)^2$$

Let's calculate the partial derivatives for our specific function f :

$$\frac{\partial f}{\partial t} = 0$$

$$\frac{\partial f}{\partial S} = \frac{1}{S_t}$$

$$\frac{\partial^2 f}{\partial S^2} = -\frac{1}{S_t^2}$$

Substituting these derivatives back into Itô's formula yields:

$$d(\ln(S_t)) = \frac{1}{S_t} dS_t + \frac{1}{2} \left(-\frac{1}{S_t^2} (dS_t)^2 \right)$$

To evaluate the squared differential term $(dS_t)^2$, let's square our original SDE expression:

$$(dS_t)^2 = (\mu S_t dt + \sigma S_t dW_t)^2$$

$$(dS_t)^2 = \mu^2 S_t^2 (dt)^2 + 2\mu\sigma S_t^2 dt dW_t + \sigma^2 S_t^2 (dW_t)^2$$

Knowing the following properties:

- $dt \cdot dt \rightarrow 0$
- $dt \cdot dW_t \rightarrow 0$
- $dW_t \cdot dW_t \rightarrow dt$

We finally get:

$$(dS_t)^2 = \sigma^2 S_t^2 dt$$

By substituting our original expression for dS_t and our newly calculated $(dS_t)^2$ into the equation, we have:

$$d(\ln(S_t)) = \frac{1}{S_t} (\mu S_t dt + \sigma S_t dW_t) - \frac{1}{2S_t^2} (\sigma^2 S_t^2 dt)$$

We distribute the terms across the fractions:

$$d(\ln(S_t)) = \mu dt + \sigma dW_t - \frac{1}{2} \sigma^2 dt$$

$$\Leftrightarrow d(\ln(S_t)) = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dW_t$$

We can directly integrate both sides from the initial time ($t = 0$) to our final maturity horizon ($t = T$):

$$\int_0^T d(\ln S_t) = \int_0^T \left(\mu - \frac{\sigma^2}{2} \right) dt + \int_0^T \sigma dW_t$$

$$\Leftrightarrow \ln(S_T) - \ln(S_0) = \left(\mu - \frac{\sigma^2}{2} \right) T + \sigma(W_T - W_0)$$

$$\Leftrightarrow \ln \left(\frac{S_T}{S_0} \right) = \left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T$$

$$\Leftrightarrow S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \right)$$

Appendix 3. — Student's t-distribution

We want to show that:

$$\lim_{\nu \rightarrow \infty} f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

. The PDF is given by:

$$f(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu} \right)^{-\frac{\nu+1}{2}}$$

If we analyze this into two independent parts, and using the fundamental definition of the exponential constant

$\lim_{n \rightarrow +\infty} (1 + \frac{a}{n})^n = e^a$, we evaluate the limit of the right-hand term:

$$\lim_{\nu \rightarrow \infty} \left(1 + \frac{x^2}{\nu} \right)^{-\frac{\nu+1}{2}} = \lim_{\nu \rightarrow \infty} \left[\left(1 + \frac{x^2}{\nu} \right)^\nu \right]^{-\frac{1}{2}} \cdot \left(1 + \frac{x^2}{\nu} \right)^{-\frac{1}{2}}$$

Thus, the variable part yields exactly the Gaussian kernel: $e^{-\frac{x^2}{2}}$

Using the Stirling's approximation for the Gamma function, which states that for large z :

$$\Gamma(z) \sim \sqrt{\frac{2\pi}{z}} \left(\frac{z}{e} \right)^z$$

Let's apply this to the ratio $\frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})}$ by setting $z = \frac{\nu}{2}$:

$$\begin{aligned} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} &\sim \frac{\sqrt{\frac{2\pi}{z+1/2}} \left(\frac{z+1/2}{e} \right)^{z+1/2}}{\sqrt{\frac{2\pi}{z}} \left(\frac{z}{e} \right)^z} \\ &= \sqrt{\frac{z}{z+1/2}} \cdot e^{-1/2} \cdot \frac{(z+1/2)^{z+1/2}}{z^z} \end{aligned}$$

Rearranging the last fraction:

$$\frac{(z+1/2)^z \cdot (z+1/2)^{1/2}}{z^z} = \left(1 + \frac{1}{2z} \right)^z \cdot \sqrt{z+1/2}$$

Taking the limit as $z \rightarrow \infty$:

- $\sqrt{\frac{z}{z+1/2}} \rightarrow 1$
- $\left(1 + \frac{1}{2z} \right)^z \rightarrow e^{1/2}$

Hence:

$$\lim_{z \rightarrow \infty} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} = \sqrt{z + \frac{1}{2}} \sim \sqrt{z} = \sqrt{\frac{\nu}{2}}$$

If we substitute this back into the normalization constant multiplier:

$$\lim_{\nu \rightarrow \infty} \frac{1}{\sqrt{\nu\pi}} \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})} = \frac{1}{\sqrt{\nu\pi}} \cdot \sqrt{\frac{\nu}{2}} = \frac{1}{\sqrt{2\pi}}$$

Combining both parts gives the standard normal density function:

$$f(x) \rightarrow \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

Let T_ν be a standard Student's t-distributed random variable with ν degrees of freedom. By definition: $\mathbb{E}[T_\nu] = 0$ and $\mathbb{V}(T_\nu) = \frac{\nu}{\nu-2}$ for $\nu > 2$.

To scale a random variable so that its variance equals 1, we must divide by its standard deviation:

$$Z = \frac{T_\nu}{\sqrt{\mathbb{V}(T_\nu)}} = \frac{T_\nu}{\sqrt{\frac{\nu}{\nu-2}}} = T_\nu \sqrt{\frac{\nu-2}{\nu}}$$

Now Z represents a zero-mean, unit-variance process containing the heavy tails dictated by ν .

Under continuous geometric scaling over a time horizon T , with an expected annualized return μ and annualized volatility

σ , the total return volatility scales by $\sqrt{T}$. We inject Z into the return dynamics:

$$\ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma\sqrt{T}Z$$

$$\Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma\sqrt{T}\left(T_\nu\sqrt{\frac{\nu-2}{\nu}}\right)$$

$$\Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) = \mu T + \sigma\sqrt{\frac{\nu-2}{\nu}}T_\nu\sqrt{T}$$

$$\Leftrightarrow S_T = S_0 \exp\left(\mu T + \sigma\sqrt{\frac{\nu-2}{\nu}}T_\nu\sqrt{T}\right)$$

Appendix 4. — Python code

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Created on Sun June 7 15:03:00 2026
5
6 @author: Quantitative Risk Management Framework
7 """
8
9 import numpy as np
10 import scipy.stats as stats
11 import matplotlib.pyplot as plt
12 import yfinance as yf
13
14 # Set global seed for exact reproducibility across all
15 # random generations
16 np.random.seed(42)
17
18 # =====
19 # 1. MARKET DATA ACQUISITION & PORTFOLIO CONSTRUCTION
20 # =====
21 print("FETCHING REAL-WORLD MARKET DATA VIA YFINANCE...")
22
23 # Define tickers for the requested global indices
24 # ^GSPC: S&P 500, ^IXIC: NASDAQ, ^FCHI: CAC 40, ^FTSE:
25 # FTSE 100, ^GDAXI: DAX, ^HSI: Hang Seng
26 tickers = {
27     'S&P 500': '^GSPC',
28     'NASDAQ': '^IXIC',
29     'CAC 40': '^FCHI',
30     'FTSE 100': '^FTSE',
31     'DAX': '^GDAXI',
32     'Hang Seng': '^HSI'
33 }
34
35 # Fetch 5 years of historical data to capture full market
36 # cycles/regimes
37 start_date = "2021-06-01"
38 end_date = "2026-06-01"
39
40 data = yf.download(list(tickers.values()),
41                     start=start_date, end=end_date)['Close']
42 data = data.dropna() # Drop non-overlapping
43 # holidays/weekends across global exchanges
44
45 # Compute daily log returns for statistical symmetry
46 asset_returns = np.log(data / data.shift(1)).dropna()
47
48 # Construct an equally-weighted portfolio
49 num_assets = len(tickers)
50 weights = np.array([1.0 / num_assets] * num_assets)
51
52 # Generate the historical portfolio returns time series
53 historical_returns = asset_returns.dot(weights).to_numpy()
54 n_days = len(historical_returns)
55
56 # Extract empirical parameters for log printout
57 emp_drift = np.mean(historical_returns)
58 emp_volatility = np.std(historical_returns)
59
60 # =====
61 # 2. USER-DEFINED SIMULATION INPUT PARAMETERS
```

```
57 # =====
58 M = 50000 # Number of Monte Carlo
59 # scenarios
60 alpha = 0.975 # Confidence level for baseline
61 # CDF plotting (Plot 2)
62
63 initial_price = 100 # Starting asset price for
64 # trajectory paths simulation
65 n_path_steps = 100 # Horizon length (in days) to
66 # project the sample paths
67 n_paths_to_plot = 15 # Number of random paths to
68 # visualize per method
69
70 print("\nINITIALIZING PORTFOLIO STRESS TESTING ENGINE...")
71 print(f"Historical Sample Size: {n_days} trading days")
72 print(f"Empirical Daily Drift (Mean): {emp_drift:.4%}")
73 print(f"Empirical Daily Volatility (Std Dev):
74 # {emp_volatility:.4%}\n" + "-"*60)
75
76 # =====
77 # APPROACH 1: GEOMETRIC BROWNIAN MOTION (NORMAL)
78 # =====
79 # GBM extracts parameters via Maximum Likelihood
80 # Estimation, assuming perfect normality
81 mu_norm, sigma_norm = stats.norm.fit(historical_returns)
82
83 # 1A. Returns for VaR/CVaR static horizon
84 sim_returns_gbm = np.random.normal(loc=mu_norm,
85                                     scale=sigma_norm, size=M)
86
87 # 1B. Dynamic path generation matrix (steps x paths)
88 paths_returns_gbm = np.random.normal(loc=mu_norm,
89                                     scale=sigma_norm, size=(n_path_steps,
90                                     n_paths_to_plot))
91 price_paths_gbm = initial_price *
92 # np.exp(np.vstack([np.zeros(n_paths_to_plot),
93 # np.cumsum(paths_returns_gbm, axis=0)]))
94
95 # =====
96 # APPROACH 2: STUDENT'S T-DISTRIBUTION
97 # =====
98 # Fits parameters directly to map history onto a
99 # theoretical t-curve (captures fat tails)
100 df_est, loc_est, scale_est =
101 # stats.t.fit(historical_returns)
102
103 # 2A. Returns for VaR/CVaR static horizon
104 sim_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
105                             scale=scale_est, size=M)
106
107 # 2B. Dynamic path generation matrix
108 paths_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
109                             scale=scale_est, size=(n_path_steps, n_paths_to_plot))
110 price_paths_t = initial_price *
111 # np.exp(np.vstack([np.zeros(n_paths_to_plot),
112 # np.cumsum(paths_returns_t, axis=0)]))
113
114 # =====
115 # APPROACH 3: KERNEL DENSITY ESTIMATION (KDE)
116 # =====
117 # Fits a non-parametric Gaussian KDE over the historical
118 # sample returns
119 kde = stats.gaussian_kde(historical_returns,
120                           bw_method='silverman')
121 h = kde.factor * np.std(historical_returns)
122
123 # 3A. Returns for VaR/CVaR static horizon (Hierarchical
124 # sampling)
125 random_indices = np.random.choice(n_days, size=M,
126                                     replace=True)
127 x_I = historical_returns[random_indices]
128 Z = np.random.normal(0, 1, M)
129 sim_returns_kde = x_I + h * Z
130
131 # 3B. Dynamic path generation matrix via structured KDE
132 # drawing loops
133 paths_returns_kde = np.zeros((n_path_steps,
134                               n_paths_to_plot))
135 for step in range(n_path_steps):
136     idx = np.random.choice(n_days, size=n_paths_to_plot,
137                             replace=True)
```



```

116 paths_returns_kde[step, :] = historical_returns[idx] +
    ↳ h * np.random.normal(0, 1, n_paths_to_plot)
117 price_paths_kde = initial_price *
    ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
    ↳ np.cumsum(paths_returns_kde, axis=0)]))
118
119 # =====
120 # RISK METRICS EVALUATION FUNCTION
121 # =====
122 def calculate_var_cvar(sim_returns, alpha=0.975):
123     """
124     Sorts returns and extracts Value at Risk and
125     ↳ Conditional Value at Risk.
126     Returns are transformed to positive numbers
127     ↳ representing losses.
128     """
129     sorted_returns = np.sort(sim_returns)
130     k = int(np.floor((1 - alpha) * len(sorted_returns)))
131
132     var = -sorted_returns[k]
133     cvar = -np.mean(sorted_returns[:k])
134     return var, cvar
135
136 # =====
137 # COMPUTE SYNCHRONIZED TARGET METRICS
138 # =====
139 # Calculate specific cross-quantile targets for Plot 1 and
140 ↳ Plot 3
141 var_gbm_99, _ = calculate_var_cvar(sim_returns_gbm,
142 ↳ alpha=0.99)
143 var_t_99, _ = calculate_var_cvar(sim_returns_t,
144 ↳ alpha=0.99)
145 var_kde_99, _ = calculate_var_cvar(sim_returns_kde,
146 ↳ alpha=0.99)
147
148 _, cvar_gbm_975 = calculate_var_cvar(sim_returns_gbm,
149 ↳ alpha=0.975)
150 _, cvar_t_975 = calculate_var_cvar(sim_returns_t,
151 ↳ alpha=0.975)
152 _, cvar_kde_975 = calculate_var_cvar(sim_returns_kde,
153 ↳ alpha=0.975)
154
155 # Calculate Skewness and Excess Kurtosis across frameworks
156 skew_gbm, kurt_gbm = stats.skew(sim_returns_gbm),
157 ↳ stats.kurtosis(sim_returns_gbm)
158 skew_t, kurt_t = stats.skew(sim_returns_t),
159 ↳ stats.kurtosis(sim_returns_t)
160 skew_kde, kurt_kde = stats.skew(sim_returns_kde),
161 ↳ stats.kurtosis(sim_returns_kde)
162
163 # =====
164 # STATISTICAL TESTS FOR SKEWNESS AND KURTOSIS
165 # =====
166 def test_skewness_kurtosis(data, name):
167     """
168     Perform statistical tests for significance of skewness
169     ↳ and excess kurtosis.
170     """
171     n = len(data)
172     sample_skew = stats.skew(data)
173     sample_kurt = stats.kurtosis(data) # Excess kurtosis
174
175     if n < 4:
176         return {
177             'name': name, 'skew': sample_skew, 'kurt':
178                 ↳ sample_kurt,
179             'skew_pvalue': np.nan, 'kurt_pvalue': np.nan,
180             ↳ 'dagostino_pvalue': np.nan
181         }
182
183     se_skew = np.sqrt(6 * n * (n - 1) / ((n - 2) * (n + 1)
184     ↳ * (n + 3)))
185     se_kurt = np.sqrt(24 * n * (n - 1)**2 / ((n - 3) * (n
186     ↳ - 2) * (n + 1) * (n + 3)))
187
188     z_skew = sample_skew / se_skew if not
189     ↳ np.isnan(se_skew) else np.nan
190     z_kurt = sample_kurt / se_kurt if not
191     ↳ np.isnan(se_kurt) else np.nan
192
193     p_skew = 2 * (1 - stats.norm.cdf(abs(z_skew))) if not
194     ↳ np.isnan(z_skew) else np.nan
195
196     p_kurt = 2 * (1 - stats.norm.cdf(abs(z_kurt))) if not
197     ↳ np.isnan(z_kurt) else np.nan
198
199     dagostino_stat, dagostino_p = stats.normaltest(data)
200
201     return {
202         'name': name, 'skew': sample_skew, 'kurt':
203             ↳ sample_kurt,
204         'z_skew': z_skew, 'z_kurt': z_kurt,
205         'skew_pvalue': p_skew, 'kurt_pvalue': p_kurt,
206         'dagostino_stat': dagostino_stat,
207         ↳ 'dagostino_pvalue': dagostino_p
208     }
209
210 # Perform tests for each distribution
211 results_gbm = test_skewness_kurtosis(sim_returns_gbm, 'GBM
212 ↳ (Normal)')
213 results_t = test_skewness_kurtosis(sim_returns_t,
214 ↳ "Student's t")
215 results_kde = test_skewness_kurtosis(sim_returns_kde,
216 ↳ 'KDE')
217
218 # =====
219 # RESULTS PRINT OUT
220 # =====
221 print(f"{'Approach':<30} | {'99.0% VaR':<10} | {'97.5%
222 ↳ CVaR':<10} | {'Skewness':<9} | {'Ex. Kurt':<9}")
223 print("-" * 78)
224 print(f"{'1. Geometric Brownian (Normal)':<30} |
225 ↳ {var_gbm_99:.4%} | {cvar_gbm_975:.4%} |
226 ↳ {skew_gbm:+0.4f} | {kurt_gbm:+0.4f}")
227 print(f"{'2. Student\'s t-Distribution':<30} |
228 ↳ {var_t_99:.4%} | {cvar_t_975:.4%} | {skew_t:+0.4f}
229 ↳ | {kurt_t:+0.4f}")
230 print(f"{'3. Kernel Density Estimator':<30} |
231 ↳ {var_kde_99:.4%} | {cvar_kde_975:.4%} |
232 ↳ {skew_kde:+0.4f} | {kurt_kde:+0.4f}")
233 print("-" * 78 + "\n")
234
235 # =====
236 # PRINT STATISTICAL TEST RESULTS
237 # =====
238 print("\n" + "=" * 80)
239 print("STATISTICAL SIGNIFICANCE TESTS FOR SKEWNESS AND
240 ↳ KURTOSIS")
241 print("=" * 80)
242
243 for results in [results_gbm, results_t, results_kde]:
244     skew_sig = '***' if results['skew_pvalue'] < 0.001
245     ↳ else '**' if results['skew_pvalue'] < 0.01 else
246     ↳ ' ' if results['skew_pvalue'] < 0.05 else ' '
247     kurt_sig = '***' if results['kurt_pvalue'] < 0.001
248     ↳ else '**' if results['kurt_pvalue'] < 0.01 else
249     ↳ ' ' if results['kurt_pvalue'] < 0.05 else ' '
250     dag_sig = '***' if results['dagostino_pvalue'] < 0.001
251     ↳ else '**' if results['dagostino_pvalue'] < 0.01
252     ↳ else ' ' if results['dagostino_pvalue'] < 0.05
253     ↳ else ' '
254
255     print(f"\n{results['name']}:")
256     print(f"Skewness: {results['skew']:+0.4f} (z =
257     ↳ {results['z_skew']:+0.4f}, p-value =
258     ↳ {results['skew_pvalue']:.6f} {skew_sig})")
259     print(f"Ex. Kurtosis: {results['kurt']:+0.4f} (z =
260     ↳ {results['z_kurt']:+0.4f}, p-value =
261     ↳ {results['kurt_pvalue']:.6f} {kurt_sig})")
262     print(f"D'Agostino K^2: stat =
263     ↳ {results['dagostino_stat']:+0.4f}, p-value =
264     ↳ {results['dagostino_pvalue']:.6f} {dag_sig}")
265
266     print("\n" + "-" * 80)
267     print("Significance codes: '***' p < 0.001, '**' p < 0.01,
268     ↳ '*' p < 0.05")
269     print("=" * 80 + "\n")
270
271 global_min_tail = np.percentile(historical_returns, 0.1)
272 global_max_tail = np.percentile(historical_returns, 15)
273
274 # Global styling mapping for the comparative charts
275 colors = {'gbm': '#1f77b4', 't': '#ff7f0e', 'kde':
276 ↳ '#2ca02c'}

```

```

234 # =====
235 # PLOT 1: PROBABILITY DENSITY TAIL FOCUS
236 # =====
237 plt.figure(figsize=(10, 6))
238 plt.hist(sim_returns_gbm, bins=250, density=True,
239         ↪ alpha=0.3, color=colors['gbm'], label='GBM (Normal)')
240 plt.hist(sim_returns_t, bins=250, density=True, alpha=0.3,
241         ↪ color=colors['t'], label='Student's t')
242 plt.hist(sim_returns_kde, bins=250, density=True,
243         ↪ alpha=0.3, color=colors['kde'], label='KDE')
244
245 plt.axvline(-var_gbm_99, color=colors['gbm'],
246         ↪ linestyle=':', linewidth=2, label=f'GBM 99% VaR
247         ↪ ({var_gbm_99:.2%})')
248 plt.axvline(-var_t_99, color=colors['t'], linestyle=':',
249         ↪ linewidth=2, label=f'Student t 99% VaR
250         ↪ ({var_t_99:.2%})')
251 plt.axvline(-var_kde_99, color=colors['kde'],
252         ↪ linestyle=':', linewidth=2, label=f'KDE 99% VaR
253         ↪ ({var_kde_99:.2%})')
254
255 plt.axvline(-cvar_gbm_975, color=colors['gbm'],
256         ↪ linestyle='-', linewidth=2, label=f'GBM 97.5% CVaR
257         ↪ ({cvar_gbm_975:.2%})')
258 plt.axvline(-cvar_t_975, color=colors['t'], linestyle='-',
259         ↪ linewidth=2, label=f'Student t 97.5% CVaR
260         ↪ ({cvar_t_975:.2%})')
261 plt.axvline(-cvar_kde_975, color=colors['kde'],
262         ↪ linestyle='-', linewidth=2, label=f'KDE 97.5% CVaR
263         ↪ ({cvar_kde_975:.2%})')
264
265 plt.title("Probability Density Focus: 99% VaR vs 97.5%
266         ↪ CVaR Loss Tail (Real Data)", fontsize=13,
267         ↪ fontweight='bold')
268 plt.xlabel("Daily Portfolio Returns")
269 plt.ylabel("Density")
270 plt.xlim(global_min_tail, global_max_tail)
271 plt.legend(fontsize=9, loc='upper left')
272 plt.grid(True, alpha=0.2)
273 plt.show()
274
275 # =====
276 # PLOT 2: CUMULATIVE DISTRIBUTION FUNCTION (CDF)
277 # =====
278 plt.figure(figsize=(10, 6))
279 for data, name, col in zip([sim_returns_gbm,
280         ↪ sim_returns_t, sim_returns_kde],
281         ↪ ['GBM (Normal)', 'Student's t',
282         ↪ 'KDE'],
283         ↪ [colors['gbm'], colors['t'],
284         ↪ colors['kde']]):
285     sorted_data = np.sort(data)
286     cdf = np.arange(1, len(sorted_data) + 1) /
287         ↪ len(sorted_data)
288     plt.plot(sorted_data, cdf, label=name, color=col,
289             ↪ linewidth=2)
290
291 plt.axhline(1-0.975, color='red', linestyle='--',
292         ↪ linewidth=1.2, label='Significance Level (0.025)')
293 plt.axhline(1-0.99, color='blue', linestyle='--',
294         ↪ linewidth=1.2, label='Significance Level (0.01)')
295 plt.title("Empirical CDF Quantile Trajectory (Real Data)",
296         ↪ fontsize=13, fontweight='bold')
297 plt.xlabel("Daily Portfolio Returns")
298 plt.ylabel("Cumulative Probability")
299 plt.xlim(np.percentile(historical_returns, 0.1), 0.01)
300 plt.ylim(0, 0.10)
301 plt.legend(fontsize=10, loc='lower right')
302 plt.grid(True, alpha=0.2)
303 plt.show()
304
305 # =====
306 # PLOT 3: DIRECT METRIC COMPARISON (BAR CHART)
307 # =====
308 fig, ax = plt.subplots(figsize=(10, 6))
309 categories = ['GBM (Normal)', 'Student's t', 'KDE']
310 var_values = [var_gbm_99 * 100, var_t_99 * 100, var_kde_99
311         ↪ * 100]
312 cvar_values = [cvar_gbm_975 * 100, cvar_t_975 * 100,
313         ↪ cvar_kde_975 * 100]
314
315 x = np.arange(len(categories))
316 width = 0.35
317 rects1 = ax.bar(x - width/2, var_values, width, label='99%
318         ↪ VaR', color='#bcbd22', alpha=0.85)
319
320 rects2 = ax.bar(x + width/2, cvar_values, width,
321         ↪ label='97.5% CVaR', color='#d62728', alpha=0.85)
322
323 ax.set_title("Direct Value Comparison: 99% VaR vs 97.5%
324         ↪ CVaR (%)", fontsize=13, fontweight='bold')
325 ax.set_xticks(x)
326 ax.set_xticklabels(categories)
327 ax.set_ylabel("Loss Scale (%)")
328 ax.legend(fontsize=10, loc='upper left')
329 ax.grid(True, axis='y', alpha=0.3)
330
331 def autolabel(rects):
332     for rect in rects:
333         height = rect.get_height()
334         ax.annotate(f'{height:.2f}%',
335                 ↪ xy=(rect.get_x() + rect.get_width() /
336                 ↪ 2, height),
337                 ↪ xytext=(0, 3), textcoords="offset
338                 ↪ ↪ points",
339                 ↪ ha='center', va='bottom', fontsize=9,
340                 ↪ fontweight='bold')
341
342 autolabel(rects1)
343 autolabel(rects2)
344 plt.tight_layout()
345 plt.show()
346
347 # =====
348 # PLOTS 4, 5, 6: SIMULATED PRICE TRAJECTORIES
349 # =====
350 for paths, name, col in zip([price_paths_gbm,
351         ↪ price_paths_t, price_paths_kde],
352         ↪ ['Geometric Brownian Motion
353         ↪ ↪ (Normal)', 'Student's
354         ↪ ↪ t-Distribution', 'Kernel
355         ↪ ↪ Density Estimator (KDE)'],
356         ↪ [colors['gbm'], colors['t'],
357         ↪ colors['kde']]):
358     plt.figure(figsize=(10, 4))
359     plt.plot(paths, linewidth=1.5, color=col, alpha=0.6)
360     plt.title(f"{name} Simulated Price Paths (Real Data)",
361             ↪ fontsize=12, fontweight='bold')
362     plt.xlabel("Time Horizon (Days)")
363     plt.ylabel("Asset Price")
364     plt.grid(True, alpha=0.3)
365     plt.axhline(initial_price, color='black',
366             ↪ linestyle='--', alpha=0.5)
367     plt.show()
368
369 # =====
370 # PREPARATION FOR DISTRIBUTION OVERLAYS (PLOTS 7-9)
371 # =====
372 x_axis = np.linspace(np.min(historical_returns),
373         ↪ np.max(historical_returns), 1000)
374 normal_curve = stats.norm.pdf(x_axis, loc=mu_norm,
375         ↪ scale=sigma_norm)
376
377 # PLOT 7: HISTOGRAM - GBM
378 plt.figure(figsize=(10, 4))
379 plt.hist(sim_returns_gbm, bins=150, density=True,
380         ↪ alpha=0.6, color=colors['gbm'], edgecolor='black',
381         ↪ linewidth=0.5)
382 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
383         ↪ linewidth=2, label='Theoretical Normal Curve')
384 plt.title("Distribution of Returns - Geometric Brownian
385         ↪ ↪ Motion (Normal)", fontsize=12, fontweight='bold')
386 plt.xlabel("Simulated Daily Returns")
387 plt.ylabel("Frequency Density")
388 plt.xlim(np.percentile(historical_returns, 0.1),
389         ↪ np.percentile(historical_returns, 99.9))
390 plt.legend()
391 plt.grid(True, alpha=0.2)
392 plt.show()
393
394 # PLOT 8: HISTOGRAM - STUDENT'S T
395 plt.figure(figsize=(10, 4))
396 plt.hist(sim_returns_t, bins=150, density=True, alpha=0.6,
397         ↪ color=colors['t'], edgecolor='black', linewidth=0.5)
398 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
399         ↪ linewidth=2, label='Theoretical Normal Curve')
400 t_curve = stats.t.pdf(x_axis, df=df_est, loc=loc_est,
401         ↪ scale=scale_est)
402 plt.plot(x_axis, t_curve, color='black', linestyle='--',
403         ↪ linewidth=2, label="Fitted Student's t Curve")

```

```

352 plt.title("Distribution of Returns - Student's
    ↳ t-Distribution", fontsize=12, fontweight='bold')
353 plt.xlabel("Simulated Daily Returns")
354 plt.ylabel("Frequency Density")
355 plt.xlim(np.percentile(historical_returns, 0.1),
    ↳ np.percentile(historical_returns, 99.9))
356 plt.legend()
357 plt.grid(True, alpha=0.2)
358 plt.show()
359
360 # PLOT 9: HISTOGRAM - KDE
361 plt.figure(figsize=(10, 4))
362 plt.hist(sim_returns_kde, bins=150, density=True,
    ↳ alpha=0.6, color=colors['kde'], edgecolor='black',
    ↳ linewidth=0.5)
363 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
    ↳ linewidth=2, label='Theoretical Normal Curve')
364 plt.plot(x_axis, kde.pdf(x_axis), color='darkgreen',
    ↳ linestyle='-', linewidth=2, label='KDE Continuous Fit
    ↳ Line')
365 plt.title("Distribution of Returns - Kernel Density
    ↳ Estimator (KDE)", fontsize=12, fontweight='bold')
366 plt.xlabel("Simulated Daily Returns")
367 plt.ylabel("Frequency Density")
368 plt.xlim(np.percentile(historical_returns, 0.1),
    ↳ np.percentile(historical_returns, 99.9))
369 plt.legend()
370 plt.grid(True, alpha=0.2)
371 plt.show()

```